

Functions and Recursion in C

[DEF] Definition to Remember

A function is a self-contained, modular block of code performing a specific task. Recursion occurs when a function calls itself directly or indirectly until reaching a base termination condition.

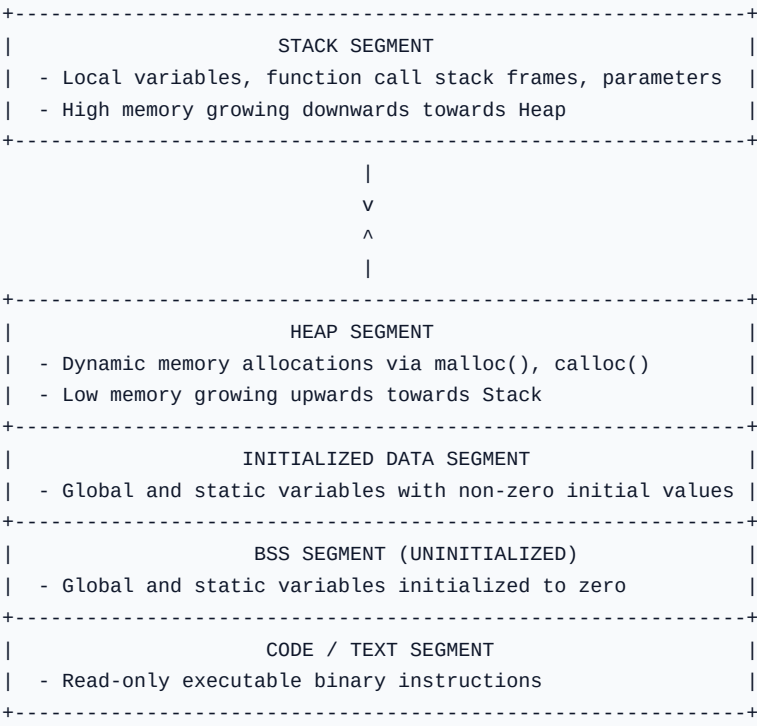
1. Concept Overview & Fundamentals

Functions and Recursion in C forms a core cornerstone of computer systems programming, hardware-level software engineering, and competitive problem solving.

In the C execution model:

2. Technical Architecture & Memory Mechanics

Memory Layout & Storage Organization



Core Architecture Breakdown

Component / Concept	Technical Specification	Operational Characteristics	Performance / Complexity
Storage Allocation	Stack vs Heap vs Data Segment	Stack is ultra-fast LIFO; Heap offers	Stack: $O(1)$ push/pop; Heap: $O(1)$
Type Checking	Static Typing	Compiler verifies types during compila	Zero runtime type penalty
Addressing Model	Byte-addressable physical RAM	Pointer stores hexadecimal byte memor	Direct memory dereference ($O(1)$)
Execution Flow	Procedural / Sequential	Controlled via jumps, branches, loops,	Deterministic instruction cycles

3. Core Syntax & Implementation Patterns

Essential Idioms & Header Declarations

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>

// Core standard implementation pattern
int main(void) {
    // Explicit initialization
    int status = 0;

    // Core logic execution
    printf("Executing C Module: Function and Recursion\n");

    return status;
}
```

4. Real-World Applications & System Engineering

- * Operating System Kernels: Linux, UNIX, and Windows NT kernels utilize C for drivers, process scheduling, and memory paging.

* En

5. Comprehensive Solved Problems & Code Walkthroughs

Problem 1: add function

Problem Description & Objective:

C Code Implementation:

Implem

```
/*
Write
param
*/

#include

// Fun
int ac
    re
}

int ma
/*
*
*
*
*
*
*
*
in
```

```
re
pr
re
}
```

Problem 2: fibonacci recursion

Problem Description & Objective:

C Code Implementation:

Implem

```
/*
Create
the Fi
*/

#include

// Tail
int fi
/*
*
*
*
*
*
*
*
*
*/
if
}
if
}
re
}

// Rec
int fi
if
}
re
}

int ma
in
pr
if
}
if
}
```

```
pr
fo
}
pr
re
}
```

Problem 3: get average function

Problem Description & Objective:

C Code Implementation:

Implement

```

/*
Call a
number
*/
#include
// Fun
float
re
}
int ma
/*
*
*
*
*
*
*
fl
av
pr
re
}

```

Problem 4: greet function

Problem Description & Objective:

C Code Implementation:

Implement

```
/*
Write
"Hello
*/
#include
```

```
// Fun
void g
    pr
}

int ma
    /*
     *
     *
     *
     *
     *
     */

    //
    gr

    re
}
```

Problem 5: increment function

Problem Description & Objective:

C Code Implementation:

Implement

```

/*
Demonstrates the origin of a variable after a function call.
*/

#include <stdio.h>

// Function to increment a variable
void increment(int *p) {
    *p = *p + 1;
}

int main() {
    /*
    *   *   *   *
    *   *   *   *
    *   *   *   *
    *   *   *   *
    */
    int x = 10;
    printf("Initial value: %d\n", x);
    increment(&x);
    printf("Value after increment: %d\n", x);
    return 0;
}

```

Problem 6: max function

Problem Description & Objective:

C Code Implementation:

Implem

```
/*
 * Create a function to find the maximum of two arguments
 */

#include <stdio.h>

// Function to find the maximum of two float numbers
float max(float a, float b)
{
    if (a > b)
        return a;
    else
        return b;
}

int main()
{
    /*
     * Example usage:
     * float a = 3.14, b = 2.71;
     * float max_val = max(a, b);
     * printf("Maximum of %f and %f is %f\n", a, b, max_val);
     */
    float a, b;
    printf("Enter two float numbers: ");
    scanf("%f %f", &a, &b);
    printf("Maximum of %f and %f is %f\n", a, b, max(a, b));
    return 0;
}
```

Problem 7: palindrome recursion

Problem Description & Objective:

C Code Implementation:

Implem

```
/*
 * Create a recursive function to check if a string is a palindrome
 */

#include <stdio.h>
#include <string.h>

// Recursive function to check if a string is a palindrome
int isPalindrome(char str[], int start, int end)
{
    if (start > end)
        return 1;
    if (str[start] != str[end])
        return 0;
    return isPalindrome(str, start + 1, end - 1);
}
```

```
    if
    }
    re
}

int ma
/*
*
*
*
*
*
*
*
in
pr
sc
re
if
}
}
re
}
```

Problem 8: print date function

Problem Description & Objective:

C Code Implementation:

Implem

```
/*
Call a
Define
*/

#inclu

// Fun
void p
pr
}

int ma
/*
*
*
*
*
*
*
*
//
```

Problem 9: square function

Problem Description & Objective:

C Code Implementation:

6. Exam Scoring Points & Memory Revision

[MUST-WRITE] Must-Write Points

- 1. Function structure: Return type, function name, parameter list, and function body.
- 2. Pass-by-value copies data to local parameters; changes do not affect caller arguments.
- 3. Pass-by-reference in C is simulated by passing pointer addresses (&var).
- 4. Recursion requirements: Base case (termination condition) and Recursive step (approaching base).
- 5. Call Stack mechanics: Each recursive call pushes an Activation Record / Stack Frame.

[RECALL] Quick Recall

Prototype Declaration -> Call Stack Push -> Parameter Binding -> Base Case Test -> Return Unwind